



SPRING-BOOT 3.5 → 4.0 · MINI GUIDE



Spring Batch: 3.5 → 4.0

Eight changes between your jobs and 4.0

One cut from the full Spring Break Cheat Sheets. Every card verified against Spring Boot 3.5.16 and 4.0.7. Real error output, real fixes, from the spring-break test suite. Steve Poole.

About this mini guide

This is one cut from **Spring Break Cheat Sheets**, the field guide to every breaking change between Spring Boot 3.5 and 4.0. Each card covers one change, verified by running real code on 3.5.16 and 4.0.7. One change, one card, one page: the error you'll see, the fix, and the traps.

HOW TO READ A CARD

TIER	How it fails. Tier 1 won't build, Tier 2 won't start, Tier 3 behaves differently. Tier 3 is the dangerous one: nothing goes red, your data just changes shape.
IMPACT	Effort per occurrence once found. S is find-and-replace, M needs an hour of care, L needs a plan.
OPENREWRITE	The badge means the Boot 4.0 recipe fixes this one for you. Run it before touching anything by hand.
WHAT YOU'LL SEE	The real output, so you can match it against your own build.
THE FIX	Before and after code, plus the steps when a rename isn't enough.
WATCH OUT	The traps that make this change look like a different bug.
VERIFY	The command that proves you're done.

THE SERIES

Spring Batch: 3.5 → 4.0	Eight changes between your jobs and 4.0	THIS GUIDE
Hibernate & JPA: 3.5 → 4.0	Ten data-layer changes, mostly the quiet kind	
Security & SSO: 3.5 → 4.0	Seven auth changes you cannot ignore	
Data & Messaging: 3.5 → 4.0	Nine changes across your stores and streams	
Testing: 3.5 → 4.0	Seven test-suite breaks, one clean build away	
Observability & Ops: 3.5 → 4.0	Six changes your dashboards will notice first	
Jackson & JSON: 3.5 → 4.0	Nine JSON changes everyone hits	
Web & HTTP: 3.5 → 4.0	Eight changes at the HTTP layer	
Core Platform & Build: 3.5 → 4.0	Twenty platform changes underneath everything	
The Big Jobs: L-Impact Changes	The six changes that eat your sprint	

The full guide collects every change in one book, with a complete planning matrix and the OpenRewrite robot list. github.com/spoole167/spring-break



About HeroDevs

HeroDevs keeps critical open source software secure after the community stops patching it. Never-Ending Support (NES) covers CVEs in Spring Boot 3.5, Java, and dozens of other end-of-life frameworks, so your team migrates on its own schedule instead of the vendor's deadline.

This book exists because migrations fail from surprise more often than difficulty. Every card is a fact, not a guess: verified by running real code on Spring Boot 3.5.16 and 4.0.7.

The complete book, every subsystem in one volume, is coming to LeanPub. Watch github.com/spoole167/spring-break for the link.

herodevs.com

The Planning Matrix

Every change in this book, in book order. Impact is the effort per occurrence once found (S small, M moderate, L large), not how many you have. Auto means the OpenRewrite Boot 4.0 recipe handles it.

8 CHANGES

6 × TIER 1 - WON'T BUILD

2 × TIER 2 - WON'T START

TIER	CHANGE	IMPACT	AUTO
1	Spring Batch ChunkHandler Renamed to ChunkRequestHandler	S	•
1	Spring Batch JobBuilder(String) Constructor Removed	S	•
1	Spring Batch 6 Listener Support Classes Removed	S	•
1	Spring Batch JobStep.setJobLauncher Removed	S	•
1	Batch Listener Base Classes Removed	M	Y
1	Spring Batch Core Package Relocations	M	•
2	Spring Batch Now Uses In-Memory Job Repository by Default	S	•
2	Spring Batch 6 Schema Sequence Rename	S	Y

The Robot List vs The Human List

The left column is what the OpenRewrite Boot 4.0 recipe fixes while you get coffee. The right column needs a person who understands your code. This split is the honest size of the migration: the renames are free, the behaviour changes are the work.

THE ROBOT LIST

Run the recipe, review the diff, done.

Batch Listener Base Classes Removed

Spring Batch 6 Schema Sequence Rename

THE HUMAN LIST

Read the card. Check your code. Decide.

Spring Batch ChunkHandler Renamed to
ChunkRequestHandler

Spring Batch JobBuilder(String) Constructor Removed

Spring Batch 6 Listener Support Classes Removed

Spring Batch JobStep.setJobLauncher Removed

Spring Batch Core Package Relocations

Spring Batch Now Uses In-Memory Job Repository by Default

TIER 1

Tier 1 - Won't Build

6 cards

1. Spring Batch ChunkHandler Renamed to ChunkRequestHandler
2. Spring Batch JobBuilder(String) Constructor Removed
3. Spring Batch 6 Listener Support Classes Removed
4. Spring Batch JobStep.setJobLauncher Removed
5. Batch Listener Base Classes Removed
6. Spring Batch Core Package Relocations

Spring Batch ChunkHandler Renamed to ChunkRequestHandler

Tier 1 - Won't Build

Spring Batch

Impact: S

ChunkHandler renamed to ChunkRequestHandler. Fails to compile on Boot 4.0.

WHAT CHANGED

`org.springframework.batch.integration.chunk.ChunkHandler` has been renamed to `ChunkRequestHandler`.

WHY

The rename of `ChunkHandler` clarifies its role in the request/response model of remote chunking.

SCOPE CHECK

Search for `ChunkHandler` (not `ChunkRequestHandler`) across all Java/Kotlin sources. Affects remote chunking setups.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/BatchRenamingUsage.java:[4,65]
error: cannot find symbol
  symbol: class ChunkHandler
  location: package org.springframework.batch.integration.chunk
```

THE FIX

ChunkHandler rename

BEFORE

```
import org.springframework.batch.integration.chunk.ChunkHandler;
// ...
ChunkHandler handler = ...;
```

AFTER

```
import org.springframework.batch.integration.chunk.ChunkRequestHandler;
// ...
ChunkRequestHandler handler = ...;
```

HOW TO FIX

Rename ChunkHandler to ChunkRequestHandler.

Update the import and all references. Only the name changed; the interface contract is the same.

FURTHER INFO

ChunkHandler belongs to the remote-chunking integration module. Verified in the same spring-break test module as the companion `JobStep.setJobLauncher` removal: the two renames fail in the same compile run but are otherwise unrelated.

spring-break module: batch-chunkhandler-renamed Spring Batch 6.0 Migration Guide

Spring Batch JobBuilder(String) Constructor Removed

Tier 1 - Won't Build

Spring Batch

Impact: S

`JobBuilder(String)` is removed. `JobRepository` is now mandatory at construction: `new JobBuilder("name", jobRepository)`.

WHAT CHANGED

`new JobBuilder(String name)` and the subsequent `.repository(JobRepository)` method have been removed. `JobRepository` is now mandatory at construction time: `new JobBuilder(String name, JobRepository repository)`. The same applies to `StepBuilder` and other builder classes.

WHY

Spring Batch 6.0 made `JobRepository` non-optional throughout the framework to ensure jobs are always backed by a persistent or in-memory store. Deferring the repository assignment was a source of subtle bugs where jobs ran without persistence.

SCOPE CHECK

Search for `new JobBuilder(`, `new StepBuilder(`, and `.repository(` across all `@Configuration` classes that define Batch jobs and steps.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/JobBuilderUsage.java:[9,33]
error: no suitable constructor found for JobBuilder(java.lang.String)
```

THE FIX

Before — repository set after construction.

```
new JobBuilder("myJob").repository(jobRepository).start(step).build();
```

After — repository required at construction

```
new JobBuilder("myJob", jobRepository).start(step).build();
```

Before @Bean method pattern

```
@Bean public Job myJob(Step step) { return new JobBuilder("myJob").start(step).build(); }
```

After @Bean method pattern

```
@Bean public Job myJob(JobRepository jobRepository, Step step) { return new JobBuilder("myJob",
jobRepository).start(step).build(); }
```

HOW TO FIX

Inject JobRepository and pass it to the constructor.

Add `JobRepository` as a parameter to your `@Bean` method (Spring will inject it) and pass it as the second argument to `new JobBuilder(...)`. The same change is needed for `StepBuilder` and any other builder that previously accepted only a name.

WATCH OUT

The `.repository()` method is also gone, so adding the repository via the fluent chain won't compile either.

Pass it in the constructor; there is no fallback.

FURTHER INFO

Spring Batch 6.0 makes the JobRepository compulsory framework-wide; it can no longer be deferred or omitted anywhere.

spring-break module: batch-job-builder-string-constructor [Spring Batch 6.0 Migration Guide](#)

Spring Batch 6 Listener Support Classes Removed

Tier 1 - Won't Build

Spring Batch

Impact: S

`JobExecutionListenerSupport`, `StepExecutionListenerSupport`, and `ChunkListenerSupport` are removed in Spring Batch 6.0. Code extending them fails to compile on Boot 4.0 with "cannot find symbol".

WHAT CHANGED

Spring Batch 6.0 (shipped with Spring Boot 4.0) removed three convenience base classes that had been deprecated since Batch 5.0: `JobExecutionListenerSupport`, `StepExecutionListenerSupport`, and `ChunkListenerSupport`. These classes provided empty default implementations of listener methods.

WHY

Java 8 interface default methods eliminate the need for adapter-style base classes. Batch 5.0 deprecated them with that rationale; Batch 6.0 follows through with removal. Implementing the listener interface directly is both more explicit and compatible with composition rather than inheritance.

SCOPE CHECK

Search for `extends` `JobExecutionListenerSupport`, `extends` `StepExecutionListenerSupport`, and `extends` `ChunkListenerSupport` in your source. Also check Spring XML configuration files and any reflection-based lookups for these class names.

WHAT YOU'LL SEE

```
// Boot 3.5 (Batch 5.x): classes present, deprecated since 5.0; compiles fine.

// Boot 4.0 (Batch 6.0): classes removed; compilation fails.
$ mvn clean compile
[ERROR] COMPILATION ERROR :
[ERROR] MyJobListener.java:[4,47] cannot find symbol
    symbol: class JobExecutionListenerSupport
[INFO] BUILD FAILURE
```

THE FIX

Replace base class with direct interface implementation

BEFORE

```
public class MyJobListener extends JobExecutionListenerSupport {
    @Override
    public void afterJob(JobExecution jobExecution) {
        // only override what you need
    }
}
```

AFTER

```
public class MyJobListener implements JobExecutionListener {
    @Override
    public void afterJob(JobExecution jobExecution) {
        // implement the interface directly
    }
}
```

HOW TO FIX

Implement the listener interface directly.

Replace `extends JobExecutionListenerSupport` with `implements JobExecutionListener`. Do the same for `StepExecutionListenerSupport` → `StepExecutionListener` and `ChunkListenerSupport` → `ChunkListener`. Implement only the methods you use: the rest have default no-op implementations on the interface.

WATCH OUT

The compiler only catches direct imports and `extends` clauses. Listeners referenced by class name in Spring XML config or loaded via reflection bypass the compiler and fail only when that code path runs. Search configuration files as well as source.

FURTHER INFO

Complements `batch-listener-classes`: both modules demonstrate the same underlying Batch 6.0 removal.

spring-break module: `batch-job-serialisation` [Spring Batch 6.0 Migration Guide](#)

Spring Batch JobStep.setJobLauncher Removed

Tier 1 - Won't Build

Spring Batch

Impact: S

JobStep.setJobLauncher replaced by setJobOperator. Fails to compile on Boot 4.0.

WHAT CHANGED

`JobStep.setJobLauncher(JobLauncher)` has been replaced by `JobStep.setJobOperator(JobOperator)`.

WHY

`setJobLauncher` was removed as part of the broader move away from `JobLauncher` toward `JobOperator` as the public API for job execution control.

SCOPE CHECK

Search for `setJobLauncher` across all Java/Kotlin sources. Affects custom job step configuration.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/BatchRenamingUsage.java:[18,19]
error: cannot find symbol
symbol:   method setJobLauncher(JobLauncher)
```

THE FIX

JobStep launcher → operator

BEFORE

```
jobStep.setJobLauncher(jobLauncher);
```

AFTER

```
jobStep.setJobOperator(jobOperator);
```

HOW TO FIX

Replace `setJobLauncher` with `setJobOperator`.

Inject a `JobOperator` bean instead of `JobLauncher` and pass it to `setJobOperator()`. `JobOperator` is available as a Spring-managed bean when Spring Batch autoconfiguration is active.

WATCH OUT

`JobLauncher` itself still exists. Only its use in `JobStep` was removed; other usages are unaffected.

FURTHER INFO

Verified in the same spring-break test module as the companion `ChunkHandler` rename: the two renames fail in the same compile run but are otherwise unrelated.

Batch Listener Base Classes Removed

Tier 1 - Won't Build

Spring Batch

Impact: M

🔄 OpenRewrite

WHAT CHANGED

Spring Batch 6.0 deleted the `*ListenerSupport` abstract classes: `JobExecutionListenerSupport`, `StepExecutionListenerSupport`, `ChunkListenerSupport`, `ItemReadListenerSupport`, and others. Listeners must now implement the interfaces directly, which have had default methods since Batch 5.0.

WHY

Java 8 default methods made the support classes unnecessary: the interfaces provide no-op defaults for every callback, so there's nothing left to inherit.

SCOPE CHECK

Search for `ListenerSupport` and `extends.*ListenerSupport` across your batch job classes. Common hits include `JobExecutionListenerSupport`, `StepExecutionListenerSupport`, and `ChunkListenerSupport`.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/batch/JobCompletionListener.java:[4,52]
error: cannot find symbol
  symbol:   class JobExecutionListenerSupport
  location: package org.springframework.batch.core.listener
[ERROR] /src/main/java/com/example/batch/StepLoggingListener.java:[4,52]
error: cannot find symbol
  symbol:   class StepExecutionListenerSupport
```

THE FIX

Job listener

BEFORE

```
import org.springframework.batch.core.listener.JobExecutionListenerSupport;

public class JobCompletionListener extends JobExecutionListenerSupport {
    @Override
    public void afterJob(JobExecution jobExecution) {
        log.info("Job completed: {}", jobExecution.getStatus());
    }
}
```

AFTER

```
import org.springframework.batch.core.JobExecutionListener;

public class JobCompletionListener implements JobExecutionListener {
    @Override
    public void afterJob(JobExecution jobExecution) {
        log.info("Job completed: {}", jobExecution.getStatus());
    }
}
```

Step listener

BEFORE

```
import org.springframework.batch.core.listener.StepExecutionListenerSupport;

public class StepLogger extends StepExecutionListenerSupport {
```

AFTER

```
import org.springframework.batch.core.StepExecutionListener;

public class StepLogger implements StepExecutionListener {
```

HOW TO FIX

Change extends to implements.

Replace `extends JobExecutionListenerSupport` with `implements JobExecutionListener`. Same for all other `*ListenerSupport` classes. The interfaces have default methods, so you only need to override the callbacks you use.

Use @BeforeJob / @AfterJob annotations.

Spring Batch also supports annotation-based listeners. Annotate methods with `@BeforeJob`, `@AfterJob`, `@BeforeStep`, `@AfterStep` etc. on any POJO; no interface needed.

WATCH OUT

If your listener extends a support class and calls `super.afterJob()` or `super.beforeStep()`, remove those super calls. The interface default methods are no-ops, but calling `super` on an interface method looks wrong and may confuse static analysis tools.

The `ItemWriter` and `ItemProcessor` interfaces also changed in Batch 6.0. If you're fixing listeners, audit the whole batch configuration in the same pass.

FURTHER INFO

The `*ListenerSupport` classes were deprecated in Spring Batch 5.0 when the interfaces gained default methods. See also: [batch-schema-rename](#).

spring-break module: [batch-job-serialisation](#) [Spring Batch 6 migration guide](#)

Spring Batch Core Package Relocations

Tier 1 - Won't Build

Spring Batch

Impact: M

WHAT CHANGED

Core domain classes have moved from `org.springframework.batch.core` to subpackages, primarily `org.springframework.batch.core.job`. Affected classes include `Job`, `JobExecution`, `JobInstance`, and `JobParameters` among others.

WHY

Spring Batch 6.0 redesigned the domain model to give each concern its own subpackage and make the module boundaries clear.

SCOPE CHECK

Search for `import org.springframework.batch.core` across all Java/Kotlin sources. Any import that resolves directly to a class in the root `core` package is likely affected.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/BatchCoreUsage.java:[3,44]
  error: package org.springframework.batch.core does not contain class Job
[ERROR] /src/main/java/com/example/BatchCoreUsage.java:[4,44]
  error: package org.springframework.batch.core does not contain class JobExecution
```

THE FIX

Core domain imports

BEFORE

```
import org.springframework.batch.core.Job;
import org.springframework.batch.core.JobExecution;
import org.springframework.batch.core.JobInstance;
import org.springframework.batch.core.JobParameters;
```

AFTER

```
import org.springframework.batch.core.job.Job;
import org.springframework.batch.core.job.JobExecution;
import org.springframework.batch.core.job.JobInstance;
import org.springframework.batch.core.job.JobParameters;
```

HOW TO FIX

Update imports to include the job subpackage.

Add `.job` to the package path for the affected classes. This is a mechanical find-and-replace:

`org.springframework.batch.core.Job` becomes `org.springframework.batch.core.job.Job`, and so on. Check the Spring Batch 6.0 Migration Guide for the complete list of moved classes.

WATCH OUT

Not all classes in `org.springframework.batch.core` moved. Some remain at the root level. Verify each import against the 6.0 Javadoc rather than bulk-replacing all root-level imports.

FURTHER INFO

The classes are unchanged; only their packages moved. Expect a large batch of import fixes if your code uses Batch domain types directly.

spring-break module: batch-package-moves [Spring Batch 6.0 Migration Guide](#)

TIER 2

Tier 2 - Won't Start

2 cards

1. Spring Batch Now Uses In-Memory Job Repository by Default
2. Spring Batch 6 Schema Sequence Rename

Spring Batch Now Uses In-Memory Job Repository by Default

Tier 2 - Won't Start

Spring Batch

Impact: S

Spring Batch defaults to an in-memory job repository in Boot 4.0, so `BATCH_JOB_EXECUTION` and related tables are no longer created. Add `spring-boot-starter-batch-jdbc` to restore database persistence.

WHAT CHANGED

Spring Boot 4.0 introduced `spring-boot-starter-batch-jdbc` as the explicit opt-in for database-backed job execution persistence. `spring-boot-starter-batch` now configures an in-memory `JobRepository` backed by a `ConcurrentHashMap`. The Batch schema DDL is no longer applied automatically unless the JDBC starter is on the classpath.

WHY

Many Batch use cases (short-lived jobs, unit tests, development) do not need database persistence, yet the old default forced every Batch application to carry a datasource and manage schema migrations. The in-memory default is the right starting point; persistence is an explicit opt-in.

SCOPE CHECK

Check for code that queries Batch tables directly via JDBC, uses `JobExplorer` to inspect execution history, or relies on Batch restart semantics (same job parameters, incomplete executions). All of these require the JDBC starter.

WHAT YOU'LL SEE

```
// Boot 3.5: BATCH_JOB_EXECUTION table created, job persisted.

// Boot 4.0 with only spring-boot-starter-batch:
// No schema created. Query against batch tables fails:
org.springframework.jdbc.BadSqlGrammarException:
  PreparedStatementCallback; bad SQL grammar
  [SELECT JOB_INSTANCE_ID, JOB_NAME from BATCH_JOB_INSTANCE ...];
  Table "BATCH_JOB_INSTANCE" not found

// Or job restart logic ignores execution history.
```

THE FIX

pom.xml — add the JDBC starter to restore database persistence

BEFORE

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-batch</artifactId>
</dependency>
```

AFTER

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-batch</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-batch-jdbc</artifactId>
</dependency>
```

HOW TO FIX

Add spring-boot-starter-batch-jdbc for database-backed persistence.

If your application relies on job execution history, restartability, or the Batch admin tables, add `spring-boot-starter-batch-jdbc` alongside `spring-boot-starter-batch`. This restores the Boot 3.5 behaviour.

Embrace the in-memory default if persistence isn't needed.

For stateless or ephemeral jobs (one-shot CLI jobs, test fixtures, data imports) the in-memory repository is correct and removes the need for a datasource. Just remove any schema initialisation configuration.

VERIFY

Batch jobs persist execution history to the database when `spring-boot-starter-batch-jdbc` is declared

WATCH OUT

This interacts with the `batch-schema-change` break:
Boot 4.0 also renamed the schema sequence from `BATCH_STEP_EXECUTION_SEQ` to `BATCH_STEP_EXECUTION_SEQUENCE`. If you add `spring-boot-starter-batch-jdbc`, make sure your schema migration handles both changes.

FURTHER INFO

In Boot 3.5, `spring-boot-starter-batch` with a JDBC datasource created the Batch schema and persisted job executions automatically. Code that relies on the Batch tables existing (auditing, restart logic, admin UI) breaks at runtime.

spring-break module: batch-in-memory-default Spring Boot 4.0 Migration Guide

Spring Batch 6 Schema Sequence Rename

Tier 2 - Won't Start

Spring Batch

Impact: S

OpenRewrite

WHAT CHANGED

Spring Batch 6 renamed the database sequences used for generating IDs: `BATCH_JOB_SEQ` became `BATCH_JOB_INSTANCE_SEQ`, `BATCH_JOB_EXECUTION_SEQ` stayed the same, and `BATCH_STEP_EXECUTION_SEQ` stayed the same. The metadata table structures also changed. Applications with existing Batch schema from Spring Batch 5 will fail when the framework tries to use the new sequence names.

WHY

The rename aligns sequence names with the tables they serve, making the schema self-documenting. The old generic names like `BATCH_JOB_SEQ` were ambiguous about which table's IDs they generated.

SCOPE CHECK

Check if your application uses `spring-boot-starter-batch` and has a persistent job repository (not in-memory). If you see `BATCH_JOB_INSTANCE`, `BATCH_JOB_EXECUTION`, or `BATCH_STEP_EXECUTION` tables in your database, you need this migration. In-memory job repositories (`MapJobRepositoryFactoryBean`) are not affected.

WHAT YOU'LL SEE

```
org.springframework.dao.InvalidDataAccessResourceUsageException:
    could not fetch next sequence value
...
Caused by: java.sql.SQLSyntaxErrorException:
    sequence "BATCH_JOB_INSTANCE_SEQ" does not exist
    at org.postgresql.core.v3.QueryExecutorImpl.receiveErrorResponse(...)
    ...
org.springframework.batch.core.launch.JobLaunchException:
    Could not launch job 'importCustomersJob'.
    Failed to create new JobInstance for job [importCustomersJob]
```

THE FIX

SQL migration script — PostgreSQL

BEFORE

```
-- Old sequences from Spring Batch 5
```

AFTER

```
-- Rename sequences to match Spring Batch 6 expectations
ALTER SEQUENCE BATCH_JOB_SEQ RENAME TO BATCH_JOB_INSTANCE_SEQ;
```

SQL migration script — MySQL

BEFORE

```
-- Old sequences from Spring Batch 5
```

AFTER

```
-- MySQL uses tables for sequence emulation  
RENAME TABLE BATCH_JOB_SEQ TO BATCH_JOB_INSTANCE_SEQ;
```

HOW TO FIX

Run a schema migration script.

Create a Flyway or Liquibase migration that renames the sequences. For PostgreSQL: `ALTER SEQUENCE BATCH_JOB_SEQ RENAME TO BATCH_JOB_INSTANCE_SEQ;` . For MySQL, rename the emulation tables. Apply this before deploying the upgraded application.

Let Spring Batch recreate the schema.

If your Batch job data is disposable (e.g., dev/staging), set `spring.batch.jdbc.initialize-schema=always` to let Spring Batch drop and recreate the metadata tables with the new names. Do not use this in production.

Use the migration SQL from Spring Batch.

The Spring Batch 6 migration guide provides official SQL scripts for each database platform. Use those as the basis for your migration rather than writing them by hand.

VERIFY

Spring Batch job launches with no BATCH_ table not found errors

WATCH OUT

Sequences are only part of the schema change. Column types and table structures also changed in Spring Batch 6. Check the official migration SQL scripts for the full set of changes.

H2 in-memory databases used in tests auto-create the schema fresh each run, so tests may pass while production fails. Test against a real database with existing Batch tables.

If you run multiple application versions against the same database during a rolling deploy, the old version will break when the sequences are renamed. Plan a maintenance window or use blue-green deployment.

FURTHER INFO

The sequence rename was announced in the Batch 6.0 milestone notes. See also: [batch-listener-classes](#).

spring-break module: [batch-schema-change](#)

[Spring Batch 6 migration guide](#)

[Spring Boot 4.0 Migration Guide](#)



Not ready to migrate?

Spring Boot 3.5 stops receiving fixes when open-source support ends. You have three sane options. Pick one deliberately.

1. **Pin 3.5 and track CVEs.** Free, but the clock is running and every new CVE is your team's problem to triage.
2. **Migrate incrementally with OpenRewrite.** The Boot 4.0 recipe automates the renames in this book; the behavioural changes still need eyes.
3. **HeroDevs NES for Spring.** Security patches for 3.5 beyond EOL, buying your team time to migrate on your schedule, not the calendar's.

Spring Boot 4.0 Migration Guide • github.com/spring-projects/spring-boot/wiki
OpenRewrite Boot 4.0 recipe • docs.openrewrite.org/recipes/java/spring/boot4
HeroDevs NES for Spring • herodevs.com/support/spring-nes
The test suite behind this book • github.com/spoole167/spring-break